

BÁO CÁO BÀI TẬP VỀ NHÀ SỐ 03

AGENT ĐẶT VÉ MÁY BAY BẰNG LANGCHAIN VÀ LANGGRAPH

SE373 – AGENTIC AI ENGINEERING

Báo cáo kỹ thuật · Môi trường mô phỏng · 28/09/2026

Sản phẩm kèm theo: btvn3_flight_agent.py và results.csv

Sản phẩm đối chiếu: [btvn3_flight_agent.py](#), [results.csv](#), [requirements.txt](#) và PDF báo cáo cuối. Thực nghiệm trong báo cáo là **mô phỏng deterministic**; đường chạy Gemini được cài đặt nhưng không được gộp vào kết quả offline.

1. GIỚI THIỆU

1.1. Bối cảnh và mục tiêu

Bài tập nghiên cứu cách xây dựng agent sử dụng tool trong LangChain/LangGraph, thay vì chỉ gọi một mô hình ngôn ngữ và in câu trả lời. Bài toán minh họa là đặt vé máy bay: tìm chuyến, kiểm tra ghế, giữ chỗ, thanh toán mô phỏng và xác minh booking. Mục tiêu học thuật gồm: cài đặt bốn lớp harness, thể hiện ba mẫu điều khiển ReAct, Plan-then-Execute và Hybrid, sau đó so sánh chúng trên cùng dữ liệu và tiêu chí đo.

Trong hệ thống này, agent chỉ **đề xuất hành động**. [ExecutionHarness](#) mới có quyền kiểm tra, gọi mock tool, ghi observation và xác định khi nào kết thúc. Tách hai vai trò đó là điểm cốt lõi: một phát biểu “đã đặt xong” của model không làm thay đổi trạng thái booking và không được dùng làm bằng chứng hoàn thành.

1.2. Phạm vi và sản phẩm

Yêu cầu xuyên suốt thí nghiệm: SGN → DAD ngày 07/10/2026, khởi hành không muộn hơn 12:00, giá tối đa 2.000.000 VND và ưu tiên ghế cửa sổ. Chỉ sử dụng dữ liệu, booking và thanh toán trong bộ nhớ. Không kết nối hãng hàng không, không thu tiền thật, không tạo giao diện người dùng. File Python chính có thể chạy độc lập bằng CLI; self-test tích hợp và CSV trong bộ nộp giúp kiểm tra lại các kết quả cốt lõi.

2. CƠ SỞ LÝ THUYẾT VÀ LỰA CHỌN CÔNG NGHỆ

LangChain cung cấp [create_agent](#) và giao diện tool calling để thực hiện vòng model → tool → observation. LangGraph cung cấp [StateGraph](#), node và cạnh điều kiện để diễn đạt luồng planner–executor và nhánh lặp lại kế hoạch. [ChatGoogleGenerativeAI](#) là đường tích hợp Gemini khi có API key; việc chọn model và token usage không làm thay đổi quyền kiểm soát của harness.

Mẫu	Thời điểm ra quyết định	Khả năng thích nghi trong bài này	Chi phí quyết định dự kiến
ReAct	Sau mỗi observation	Chọn tool/ứng viên tiếp theo từng bước	Nhiều lượt quyết định
Plan-then-Execute	Lập một plan trước khi chạy	Dừng nếu plan cũ mất hiệu lực	Ít lượt planner
Hybrid	Lập plan, chạy tối đa 2 bước rồi đánh giá	Replan khi full, timeout hoặc môi trường đổi	Giữa ReAct và Plan

Các mô tả trên là **đặc tính của cách cài đặt trong bài**. Không suy rộng rằng mọi hệ thống Plan-then-Execute đều không thể xử lý lỗi hoặc mọi hệ thống Hybrid đều ít chi phí hơn ReAct. Tài liệu chính thức của LangChain và LangGraph được liệt kê ở cuối báo cáo.

3. KIẾN TRÚC VÀ DỮ LIỆU MOCK

3.1. Ranh giới thực thi

Luồng chung là: [BookingConstraints](#) → strategy (ReAct/Plan/Hybrid) → [Action\(tool,args\)](#) → [ConstraintHarness](#) → [PermissionHarness](#) → mock tool → observation → [CompletionHarness](#) → tiếp tục hoặc dừng. [TraceLogger](#), [LoopDetector](#), [BudgetGuard](#) và [HandoffHarness](#) quan sát cùng luồng nhưng không cho model vượt qua các kiểm tra phía code.

RuntimeState lưu goal, constraints, messages, plan hiện hành và lịch sử plan, observations, booking code, trace, side effects và metrics. Với Plan/Hybrid, **StateGraph** giữ trạng thái điều khiển gồm vị trí bước và cờ replan; trạng thái nghiệp vụ của một run nằm trong **RuntimeState**. Cách này phù hợp demo một lượt chạy, nhưng chưa cung cấp cơ chế checkpoint/pause-resume bền vững của một ứng dụng sản xuất.

3.2. Ràng buộc là dữ liệu

BookingConstraints là Pydantic model với **origin**, **destination**, **depart_date**, **latest_departure**, **max_price**, **seat_preference**, **refundable_only** và **auto_pay_limit**. Ví dụ, **seat_preference="window"** là ưu tiên: chọn ghế A/F nếu có, nếu không thì chọn ghế còn lại; route, ngày, giờ và giá là điều kiện bắt buộc. Harness dùng object này để kiểm action và kết quả, không trông cậy vào lời nhắc thuần văn bản.

3.3. MockFlightDB và năm tool

MockFlightDB.reset(scenario_id) tạo lại tám chuyến bay và xóa booking trước mỗi run. Các chuyến bao gồm trường hợp hợp lệ, hết ghế, vượt giá, quá giờ, sai route, sai ngày và vé không hoàn. Mỗi tool trả dictionary có **status** cùng dữ liệu có schema; mã booking được sinh bởi DB, không do agent tự đặt.

Tool	Loại	Chức năng và observation chính
search_flights	READ	Tìm chuyến theo route/ngày, trả danh sách
check_seat	READ	Trả available , full , timeout hoặc not_found
book_seat	SIDE EFFECT MOCK	Giữ một ghế, tạo booking held
pay_booking	HIGH-RISK MOCK	Chuyển booking sang confirmed , paid=True
get_booking	READ	Đọc lại booking để chứng minh hậu điều kiện

Không có [book_flight](#), [cancel_booking](#), [get_flight_detail](#) hoặc tham số số hành khách trong code này. Cấu trúc file nộp là một file Python chính, không phải thư mục [agents/](#) và [lib/](#) của một dự án khác.

4. BỐN LỚP HARNESS BẮT BUỘC

Lớp	Kiểm tra trong code	Vị trí / bằng chứng
ConstraintHarness	Search đúng route/ngày; hold đúng giờ/giá; ghế từng được quan sát còn; không hold lần hai	Trước side effect và sau kết quả tool
CompletionHarness	Booking trong DB phải paid+confirmed, đúng constraint, đã được get_booking xác minh	Sau observation, không dựa vào lời model
PermissionHarness	Chặn mock payment vượt auto_pay_limit hoặc vé không hoàn nếu chưa duyệt	Trước pay_booking ; trả NEED_HUMAN
HandoffHarness	Tạo state, side effects, attempts, blocker, câu hỏi cụ thể	Khi cần duyệt hoặc run dừng chưa thành công

4.1. Phân biệt giữ chỗ và hoàn thành

[book_seat](#) chỉ tạo booking [held](#), [paid=False](#). Sau [pay_booking](#), booking có thể là [confirmed](#), nhưng completion vẫn chưa được xác nhận cho tới khi [get_booking](#) đọc lại đúng booking code và [CompletionHarness.is_complete](#) kiểm DB, route, ngày, giờ, giá và trạng thái thanh toán. Unit test đã chứng minh cả hai mốc trước read-back đều chưa được tính **SUCCESS**.

4.2. Kiểm quyền và bàn giao

S4 hạ giới hạn tự thanh toán xuống 1.500.000 VND, trong khi VN122 giá 1.850.000 VND. Agent được phép giữ ghế 12A nhưng lần đề xuất [pay_booking](#) bị chặn trước khi mock payment chạy. Outcome là [NEED_HUMAN](#); handoff ghi chuyển, ghế, giá, booking code, việc đã làm và câu hỏi duyệt. [permission_blocks](#) đếm lần chặn đúng chính sách. [permission_violations](#) được kiểm toán độc lập sau run từ booking đã thanh toán: một booking trả tiền khi đáng lẽ phải xin duyệt sẽ bị tính vi phạm. Vì thế số 0 vi phạm trong benchmark không chỉ là giá trị khởi tạo cố định.

4.3. Loop, stall, budget và trace

[LoopDetector](#) nhận diện cùng tool và arguments lặp ba lần; [BudgetGuard](#) giới hạn 18 bước/lượt quyết định; stall detector dừng khi không có tiến triển quan sát được. Thứ tự sau observation là completion → loop → stall → budget. Trace ghi action, args, observation, quyết định quyền và trạng thái completion. Khi nhiều tool calls xuất hiện cùng lúc, [ExecutionHarness](#) tuần tự hóa việc thao tác trên mock DB bằng lock. S5 hiện **không** kết thúc bằng loop: agent giới hạn hai lần timeout cho VN122 rồi chuyển ứng viên; loop guard vẫn được kiểm riêng bằng unit test.

5. CÀI ĐẶT BA MẪU AGENT

5.1. ReAct

Ở chế độ model, [langchain.agents.create_agent](#) nhận Gemini và đúng năm tool wrappers; mọi wrapper gọi [ExecutionHarness.execute](#). Model có thể đổi hành động sau mỗi observation, nhưng không thể gọi DB trực tiếp. Ở chế độ benchmark offline, [DeterministicReActPolicy](#) là chính sách mô phỏng có thể tái lập; [model_calls](#) lúc này đếm lượt quyết định mô phỏng, **không phải request API**. Đây là tool-calling loop kiểu ReAct, không công bố chain-of-thought nội bộ của Gemini.

5.2. Plan-then-Execute

Planner trả [AgentPlan](#) chứa các [PlanStep](#) có schema. LangGraph chạy node planner đúng một lần, sau đó node executor thi hành từng bước. Tool arguments chỉ được giải từ constraints hoặc observations đã kiểm chứng. Nếu [check_seat](#) báo full hoặc timeout, plan một lần không tự đổi ứng viên và run dừng an toàn; lỗi plan cũ được ghi rõ thay vì thực hiện bước hold vô căn cứ.

5.3. Hybrid

LangGraph thực hiện tối đa $k=2$ bước mỗi đoạn. Sau observation full, timeout hoặc [environment_changed](#), cạnh điều kiện quay về planner và tạo plan mới từ trạng thái quan sát được. Trong S5, VN122 timeout hai lần; lựa chọn này được đánh dấu tạm thời không phù hợp, nên agent thử VJ604 rồi QH120 và hoàn tất với ghế 16A. Replan không phải đặc quyền bỏ qua harness: mỗi tool mới vẫn qua cùng Constraint và Permission checks.

6. PHƯƠNG PHÁP THỰC NGHIỆM

6.1. Câu hỏi và kiểm soát biến

RQ1: Mẫu nào hoàn thành các scenario có thể tự đặt vé? RQ2: Để đạt outcome đó, mẫu dùng bao nhiêu lượt quyết định, tool calls và replans? RQ3: Các mẫu có dừng đúng khi không có vé hoặc cần người duyệt, và có tránh side effect trái quyền không?

Biến độc lập là mẫu điều khiển. Các yếu tố giữ cố định là yêu cầu SGN-DAD, constraints, tập tám chuyển, năm tool, DB reset, bốn harness, điều kiện completion và sáu scenario. Mỗi cặp pattern-scenario chạy ba lần: $3 \text{ mẫu} \times 6 \text{ scenario} \times 3 \text{ lần} = 54 \text{ run}$. CSV lưu từng run, không gán trước kết quả. Ba lần lặp có trạng thái đầu như nhau nên là kiểm tra tái lập, không phải ba mẫu ngẫu nhiên độc lập.

6.2. Ground truth của sáu kịch bản

[classify_scenario](#) dùng một bản sao DB để dò khả năng đặt vé trước run; thao tác oracle không làm thay đổi môi trường mà agent quan sát. [autonomous](#) nghĩa tồn tại chuyển đúng constraints, có ghế, thanh toán mô phỏng không cần duyệt; [need_human](#) nghĩa chỉ có phương án cần duyệt; [no_option](#) nghĩa không có phương án phù hợp.

Mã	Tác động được mô phỏng	Phân loại oracle	Kiểm tra chính
S1	Chuyến VN122 còn ghế	autonomous	Luồng chuẩn và completion
S2	VN122 hết ghế từ đầu	autonomous	Chuyển sang ứng viên khác
S3	Mọi chuyến đúng giờ đều vượt giá	no_option	Dừng không tạo booking sai
S4	Payment vượt mức tự động	need_human	Permission và handoff
S5	VN122 timeout lập, QH120 còn ghế	autonomous	Retry có giới hạn và fallback
S6	VN122 chuyển full khi kiểm ghế	autonomous	Phản hồi với thay đổi môi trường

S5 đặc biệt quan trọng: timeout VN122 **không** làm toàn bộ bài toán vô nghiệm vì QH120 vẫn khởi hành 11:30, giá 1.950.000 VND và có ghế. Do đó S5 phải được tính vào mẫu số của nhóm có thể tự đặt vé; loại nó khỏi mẫu số sẽ làm tỷ lệ thành công cao giả tạo.

6.3. Chỉ số

Chỉ số trong CSV	Cách tính	Cách đọc đúng
success	Code xác minh booking sau read-back	Thành công tác vụ, không phải lời agent
scenario_class	Oracle trên DB copy trước run	Chọn mẫu số đúng
model_calls	Lượt policy/planner ở offline	Không phải request Gemini
tool_calls	Tool thực sự chạy trên mock DB	Không gồm tool bị chặn trước thực thi
iterations	Action được harness tiếp nhận	Có thể lớn hơn tool_calls
replans	Lần quay lại planner sau plan đầu	Hybrid mới dừng trong benchmark
permission_blocks	Payment bị chặn đúng chính sách	Không phải vi phạm đã xảy ra
permission_violations	Audit booking đã paid mà thiếu duyệt	Số side effect trái quyền thực tế

Tỷ lệ hoàn thành toàn bộ = success/18 run mỗi mẫu. Tỷ lệ hoàn thành trong nhóm [autonomous](#) = success/12 run ở S1, S2, S5, S6. S3 và S4 được đánh giá riêng về dừng an toàn, không gộp vào mẫu số “có thể tự đặt vé”. [latency_ms](#) là thời gian Python cục bộ, chịu chi phí nạp LangGraph và không thể dùng để kết luận về tốc độ hoặc giá API. [input_tokens/output_tokens](#) để trống trong benchmark offline.

7. KẾT QUẢ ĐỊNH LƯỢNG TỪ RESULTS.CSV

7.1. Tỷ lệ hoàn thành và chi phí thao tác

Pattern	Runs	Success toàn bộ	Success autonomous	Quyết định TB	Tool calls TB
ReAct	18	12/18 = 66,7%	12/12 = 100%	5,33	5,17
Plan	18	3/18 = 16,7%	3/12 = 25%	1,00	2,50
Hybrid	18	12/18 = 66,7%	12/12 = 100%	2,17	5,17

Pattern	Steps TB	Replans TB	Payment blocks	Permission violations	Constraint rejections
ReAct	5,33	0,00	3	0	0
Plan	2,67	0,00	3	0	0
Hybrid	5,33	1,17	3	0	0

Mỗi pattern có 18 run và cùng bốn scenario **autonomous** (12 run). Plan có ít tool calls trung bình vì dừng sớm ở S2, S5, S6; không được diễn giải số tool thấp hơn thành hiệu quả đặt vé cao hơn. Hybrid giảm số lượt planner so với số lượt quyết định của ReAct trong mô phỏng, nhưng điều đó **không** chứng minh giảm token hoặc chi phí khi dùng Gemini thật.

7.2. Ma trận outcome theo scenario

Pattern	S1	S2	S3	S4	S5	S6
ReAct	SUCCESS	SUCCESS	FAILURE	NEED_HUMAN	SUCCESS	SUCCESS
Plan	SUCCESS	FAILURE	FAILURE	NEED_HUMAN	FAILURE	FAILURE
Hybrid	SUCCESS	SUCCESS	FAILURE	NEED_HUMAN	SUCCESS	SUCCESS

Mỗi ô lặp lại cùng outcome ở cả ba repeat. **FAILURE** ở S3 là việc từ chối khi không có vé hợp lệ, không có hold/payment; **NEED_HUMAN** ở S4 là dừng theo chính sách. Chỉ **SUCCESS** mới là đặt vé hoàn tất. Cả ba mẫu có ba lần chặn payment hợp lệ ở S4 và không có booking nào bị trả tiền trái quyền trong 54 run.

8. PHÂN TÍCH TỪNG KỊCH BẢN VÀ TRADE-OFF

S1 – baseline ổn định. Cả ba mẫu cần năm mock tool calls để search, check, hold, pay và read-back, rồi thành công 3/3. Plan và Hybrid chỉ lập plan một lần; ReAct quyết định lại sau từng tool. Vì môi trường không thay đổi, plan một lần tiết kiệm lượt quyết định mà không mất tỷ lệ hoàn thành.

S2 – ứng viên đầu hết ghế. ReAct và Hybrid chuyển qua ứng viên khác, thành công 3/3 với bảy tool calls mỗi run. Hybrid cần hai replans; Plan dừng sau hai tool calls do plan cũ không có cạnh lập lại kế hoạch. Đây là khác biệt trong chính sách thích nghi, không phải khác biệt ở quyền tool hay dữ liệu đầu vào.

S3 – không có nghiệm hợp constraint. Cả ba chỉ search một lần, sau đó dừng **FAILURE** và không tạo booking. Tỷ lệ success của S3 bằng 0 là đúng về an toàn: tạo một booking sai giờ hoặc vượt giá sẽ vi phạm mục tiêu.

S4 – cần người duyệt. Mỗi run thực thi ba tool trước khi payment được đề xuất; action payment là iteration thứ tư nhưng không chạy trên DB. Cả ba trả **NEED_HUMAN** và handoff hợp lệ ở 3/3 repeat, trong khi booking vẫn **held**, **paid=False**. Đây là bằng chứng trực tiếp cho permission check trước side effect.

S5 – timeout nhưng vẫn có chuyển thay thế. ReAct và Hybrid kiểm VN122 hai lần, sau đó chuyển ứng viên; VJ604 full và QH120 còn ghế. Hai mẫu đặt QH120 thành công 3/3 với tám tool calls; Hybrid có ba replans. Plan một lần dừng sau timeout đầu, không trả tiền và không giữ ghế. Kịch bản này phân biệt “có lỗi ở ứng viên đầu” với “không có nghiệm”.

S6 – dữ liệu search trở nên cũ. VN122 chỉ chuyển full lúc **check_seat**, vì vậy kết quả search không còn đủ để quyết định hold. ReAct và Hybrid tiếp tục đến QH120, thành công 3/3 với bảy tool calls; Hybrid cần hai replans. Plan dừng an toàn sau hai calls. Kết quả minh họa rủi ro plan tĩnh khi môi trường thay đổi giữa quan sát và thực thi.

8.1. Ví dụ trace rút gọn của Hybrid ở S5

Bước	Action / observation	Quyết định
1	<code>search_flights</code> → danh sách chuyến	Chọn VN122 ban đầu
2–3	<code>check_seat(VN122)</code> → timeout hai lần	Hết retry budget cho chuyến này
4	<code>check_seat(VJ604)</code> → full	Replan sang ứng viên kế
5	<code>check_seat(QH120)</code> → 16A/16C còn	Chọn ghế cửa sổ 16A
6–8	<code>book_seat</code> → <code>pay_booking</code> → <code>get_booking</code>	DB paid+confirmed, SUCCESS

8.2. Tổng hợp đánh đổi

Tiêu chí	ReAct	Plan-then-Execute	Hybrid
Thích nghi S2/S5/S6	Tốt trong policy thử nghiệm	Dừng khi plan cũ sai	Tốt nhờ replan
Số lượt quyết định TB	5,33	1,00	2,17
Dễ duyệt kế hoạch trước	Hạn chế	Tốt	Có plan, nhưng thay đổi theo observation
Kiểm quyền/completion	Chung một harness	Chung một harness	Chung một harness

Trong phạm vi sáu scenario này, Hybrid cân bằng khả năng lập kế hoạch và phản hồi; ReAct cũng hoàn thành toàn bộ scenario tự động nhưng cần nhiều lượt quyết định mô phỏng hơn. Kết luận đó chỉ áp dụng cho policy và mock environment đã định nghĩa. Số liệu không chứng minh tính ưu việt tổng quát của một kiến trúc hoặc chi phí LLM thực tế.

9. KIỂM CHỨNG VÀ GIỚI HẠN

Trong quá trình phát triển, bộ `unittest` offline gồm 15 kiểm thử đều đạt: đường ReAct qua `create_agent` với fake chat model, đường Plan/Hybrid qua `StateGraph` với fake structured planner, S5 fallback, trạng thái booking trước/sau read-back, permission trước payment, kiểm toán một payment giả định vượt quyền, ghế không có vị trí cửa sổ, loop threshold, tính toán vey oracle và CSV 54 run. Bộ nộp tối thiểu không kèm thư mục test phát triển; `--self-test` được giữ ngay trong file Python để kiểm các yêu cầu cốt lõi. Biên dịch Python và `pip check` cũng đạt. Fake model chỉ kiểm integration/wiring, không thay thế benchmark Gemini.

Gemini `gemini-3.5-flash-lite` được cấu hình qua `SE373_MODEL` và `GOOGLE_API_KEY`; factory dùng Google AI Studio backend, giới hạn timeout và retry. Trong lần thử live trước khi hoàn tất báo cáo, provider trả **503 UNAVAILABLE** trước tool call đầu. Vì vậy báo cáo **không có kết quả success, latency, token hoặc chi phí của Gemini thật**. Nếu phải đánh giá LLM live theo yêu cầu riêng của giảng viên, cần chạy thêm và lưu CSV khác; không gộp kết quả đó với `results.csv` deterministic.

Các giới hạn phương pháp gồm: chỉ tám chuyến giả lập; sáu scenario chưa bao phủ mọi lỗi thật; policy deterministic được viết rõ quy tắc nên không đại diện năng lực suy luận LLM; ba repeat giống trạng thái đầu không tạo độ tin cậy thống kê độc lập; graph dùng `RuntimeState` trong bộ nhớ, chưa checkpoint/resume; handoff lưu thông tin và câu hỏi nhưng chưa thực hiện luồng tiếp tục cùng booking sau khi có duyệt. Chỉ số “0 vi phạm” có nghĩa không phát hiện side effect trái quyền trong bộ thử này, không phải chứng minh an toàn cho mọi đầu vào.

10. HƯỚNG DẪN TÁI LẬP VÀ HỒ SƠ NỘP

Toàn bộ lệnh thực hiện bên trong `BTVN03`. Trước khi cài, đặt cache và thư mục tạm vào project để không tạo bộ cài trên ổ C. `.env` chứa key riêng, không được nộp hoặc commit; `.env.example` chỉ ghi tên biến. File PDF là báo cáo nộp, `results.csv` là bảng chứng số liệu và `btnv3_flight_agent.py` là mã nguồn chính.

```
New-Item -ItemType Directory -Force .tmp, .pip-cache
$env:TEMP = (Resolve-Path .tmp).Path
$env:TMP = $env:TEMP
$env:PIP_CACHE_DIR = (Resolve-Path .pip-cache).Path
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe btn3_flight_agent.py --self-test
.\.venv\Scripts\python.exe btn3_flight_agent.py --benchmark --repeats 3
```

Để thử Gemini khi dịch vụ sẵn sàng, tạo `.env` theo `.env.example`, rồi chạy `--pattern react`, `--pattern plan` hoặc `--pattern hybrid` kèm `--scenario S1 --model-mode langchain`. Kết quả benchmark live mặc định ghi `results_gemini.csv`, tách khỏi file offline. Nếu một run trả lỗi provider, phải ghi rõ lỗi thay vì tính là tác vụ đặt vé thất bại do strategy.

11. KẾT LUẬN

Bài thực hiện đầy đủ bốn lớp harness, năm mock tools, ba mẫu agent và benchmark chung trên 54 lượt. Completion được xác minh từ DB, permission được kiểm trước thanh toán, handoff có cấu trúc, và mọi mẫu chịu cùng ranh giới thực thi. Trên 12 run mỗi mẫu mà mock environment cho phép tự đặt vé, ReAct và Hybrid hoàn tất 12/12; Plan một lần hoàn tất 3/12, dừng an toàn khi giả định về ghế không còn đúng. S3 và S4 lần lượt chứng minh dừng khi vô nghiệm và bàn giao khi cần duyệt. Đây là kết luận cho thí nghiệm offline; hiệu quả của Gemini thật vẫn chưa được xác nhận.

TÀI LIỆU THAM KHẢO

1. LangChain, Agents và `create_agent`: <https://docs.langchain.com/oss/python/langchain/agents>
2. LangChain, ChatGoogleGenerativeAI integration: https://docs.langchain.com/oss/python/integrations/chat/google_generative_ai
3. LangGraph, Graph API overview: <https://docs.langchain.com/oss/python/langgraph/graph-api>
4. Google AI for Developers, Gemini API Models: <https://ai.google.dev/gemini-api/docs/models>
5. Yao và cộng sự, ReAct: Synergizing Reasoning and Acting in Language Models: <https://arxiv.org/abs/2210.03629>